

■ 2.3 Patterns

■ 2.3.1 When Patterns Match

Section 2.2.6 introduced patterns as a way of representing *classes* of expressions in *Mathematica*.

The basic way that you construct a pattern in *Mathematica* is to take an expression, and replace pieces of it by “blanks” (written `_`). Each blank can stand for any expression.

As a result, a pattern like `f[_^_, _]` stands for a class of expressions of the form `f[anything^anything, anything]`.

You can think of this as giving the “outline” of an expression, but leaving “blanks” or “holes” in which arbitrary pieces can be inserted.

When you use patterns, particularly in transformation rules, it is often convenient to give “names” to the blanks. An object like `x_` stands for a blank with the name `x`. The important rule is that different occurrences in a particular pattern of blanks with the *same name* must stand for the *same expression*. The pattern `f[_,_]` can stand for any expression of the form `f[x, y]`, where `x` and `y` need not be the same. On the other hand, `f[x_,x_]` can *only* stand for expressions in which the two arguments of `f` are exactly the same.

This gives a rule for the value of the function `f` with two identical arguments.

```
In[1]:= f[x_, x_] := p[x]
```

The rule applies to `f[a, a]`, but not to `f[a, b]`.

```
In[2]:= f[a, a] + f[a, b]
Out[2]= f[a, b] + p[a]
```

It is important to realize that patterns in *Mathematica* stand for classes of expressions with the same *structure*. Even though two expressions may be *mathematically equal*, they cannot be represented by the same *Mathematica* pattern unless they have the same structure.

Thus, for example, the pattern `(1 + x_)^2` can stand for expressions like `(1 + a)^2` or `(1 + b^3)^2` that have the same *structure*. However, it cannot stand for the expression `1 + 2 a + a^2`. Although this expression is *mathematically equal* to `(1 + a)^2`, it does not have the same *structure* as the pattern `(1 + x_)^2`.

The fact that patterns in *Mathematica* specify the *structure* of expressions is important in making it possible for you to set up transformation rules which change the *structure* of expressions, while leaving them mathematically equal.

It is worth realizing that in general it would be quite impossible for *Mathematica* to match patterns by mathematical, rather than structural, equivalence. In the case of expressions like $(1 + a)^2$ and $1 + 2a + a^2$, you can determine equivalence just by using functions like **Expand** and **Factor**. But, as we discussed on page 260 there is in general no way to find out whether an arbitrary pair of mathematical expressions are equal.

With two identical arguments, the rule defined above applies.

```
In[3]:= f[a^2 - 1, a^2 - 1]
Out[3]= p[-1 + a^2]
```

Even though these arguments are mathematically equal, they are not structurally equivalent, so the pattern does not match.

```
In[4]:= f[a^2 - 1, (1 + a)(1 - a)]
Out[4]= f[-1 + a^2, (1 - a)(1 + a)]
```

Although *Mathematica* matches in a purely structural fashion, its notion of structural equivalence is quite sophisticated.

It considers, for example, the two expressions like $x + y$ and $y + x$ to be equivalent. As a result, a pattern like $g[x_+ + y_+, x_+]$ in *Mathematica* can match both $g[a + b, a]$ and $g[a + b, b]$.

In fact, whenever *Mathematica* encounters an *orderless* or *commutative* function such as **Plus** or **Times** in a pattern, it effectively tests all the possible orders of arguments to try and find a match. Sometimes, there may be several orderings that lead to matches. In such cases, *Mathematica* just uses the first ordering it finds. For example, $h[x_+ + y_+, x_+ + z_+]$ could match $h[a + b, a + b]$ with $x \rightarrow a, y \rightarrow b, z \rightarrow b$ or with $x \rightarrow b, y \rightarrow a, z \rightarrow a$. *Mathematica* tries the case $x \rightarrow a, y \rightarrow b, z \rightarrow b$ first, and so uses this match.

This defines a transformation rule for the pattern $g[x_+ + y_+, x_+]$.

```
In[5]:= g[x_+ + y_+, x_+] := p[x, y]
```

This expression has exactly the same form as the pattern.

```
In[6]:= g[a + b, a]
Out[6]= p[a, b]
```

Mathematica matches the pattern by rearranging this expression as $g[b + a, b]$.

```
In[7]:= g[a + b, b]
Out[7]= p[b, a]
```

Here is a more complicated definition.

```
In[8]:= h[x_+ + y_+, x_+ + z_+] := p[x, y, z]
```

This can match $h[x_+ y_+, x_+ z_+]$ either with $x \rightarrow a$ or with $x \rightarrow b$. *Mathematica* tries $x \rightarrow a$ first, and so uses this match.

```
In[9]:= h[a + b, a + b]
Out[9]= p[a, b, b]
```

In addition to being orderless, functions like **Plus** and **Times** also have the property of being *flat* or *associative*. This means that you can effectively “parenthesize” their arguments in any way, so that, for example, $x + (y + z)$ is equivalent to $x + y + z$, and so on.

Mathematica takes account of flatness in matching patterns. As a result, a pattern like $g[x_+ y_+]$ can match $g[a + b + c]$, with $x \rightarrow a$ and $y \rightarrow (b + c)$.

Mathematica uses the associativity of addition to write this expression as $g[a + (b + c), a]$, so that it has the same form as the pattern $g[x_+ y_+, x_+]$.

```
In[10]:= g[a + b + c, a]
Out[10]= p[a, b + c]
```

Mathematica uses the commutativity and associativity of addition to try several different arrangements of this expression. It finds a match when the expression is put in the form $g[(b + d) + (a + c), b + d]$.

```
In[11]:= g[a + b + c + d, b + d]
Out[11]= p[b + d, a + c]
```

This does not match the pattern.

```
In[12]:= g[a, a]
Out[12]= g[a, a]
```

This matches the pattern, since $a - b$ is treated as $a + (-1 b)$.

```
In[13]:= g[a - b, a]
Out[13]= p[a, -b]
```

The associativity of **Plus** allows a pattern like $x_+ y_+$ to match expressions like $a + b$ or $a + b + c$. The pattern cannot, however, match the object a on its own.

Nevertheless, in certain situations, it is useful to think of a as the “sum” $a + 0$, and to allow it to match a sum of terms. The special notation $x_+ y_+$ stands for a sum in which y_+ is *optionally included*. The pattern $x_+ y_+$ can match either an explicit sum of terms in which both x_+ and y_+ appear, or a single term x_+ , with y taken to be 0.

This pattern allows both sums of terms, and single terms.

```
In[14]:= k[x_+ y_+] := q[x, y]
```

This matches the pattern, with y taken to have its “default value” of 0.

```
In[15]:= k[a]
Out[15]= q[a, 0]
```

Mathematica can treat “optional arguments” like y_+ in several standard func-

tions.

$x_+ + y_-$	default for y is 0
$x_- y_-$	default is 1
$x_-^+ y_-$	default is 1

Patterns with optional pieces.

This definition applies to expressions *structurally* of the form $g[\text{anything}^{\text{anything}}]$.

```
In[16]:= g[x_^n_] := gp[x, n]
```

The pattern matches this expression.

```
In[17]:= g[a^b]
Out[17]= gp[a, b]
```

The pattern does not match this expression, even though $a+b$ could mathematically be considered as $(a+b)^1$.

```
In[18]:= g[a+b]
Out[18]= g[a + b]
```

Now the exponent is specified as being *optional*.

```
In[19]:= h[x_^n_] := hp[x, n]
```

The new pattern matches this expression.

```
In[20]:= h[a^b]
Out[20]= hp[a, b]
```

It also matches this pattern, with $n \rightarrow 1$.

```
In[21]:= h[a + b]
Out[21]= hp[a + b, 1]
```

This is a common form of pattern. $a_+ + b_+ x_+$ matches any linear function of x_+ .

```
In[22]:= u[a_+ + b_+ x_+, x_] := up[a, b]
```

In this case, $b \rightarrow 1$.

```
In[23]:= u[1 + x, x]
Out[23]= up[1, 1]
```

Here $b \rightarrow 1$ and $a \rightarrow 0$.

```
In[24]:= u[y, y]
Out[24]= up[0, 1]
```